Язык программирования C

Лекция 8: Макросы

Лисица Никита

2026

Макросы

- Макросы – инструмент текстовой замены одних выражений на другие

Макросы

- Макросы – инструмент текстовой замены одних выражений на другие
- Выполняются **препроцессором**, до основного прохода компилятора – т.е. до проверки типов, до разрешения имён, и т.п.

Макросы

- Макросы – инструмент текстовой замены одних выражений на другие
- Выполняются **препроцессором**, до основного прохода компилятора – т.е. до проверки типов, до разрешения имён, и т.п.
- Заменить можно **не любое** выражение: макросы всегда выглядят, как переменные или как функции:

```
#define PI 3.1415926535
#define SUM(a, b) a + b
```

Макросы

- Макросы – инструмент текстовой замены одних выражений на другие
- Выполняются **препроцессором**, до основного прохода компилятора – т.е. до проверки типов, до разрешения имён, и т.п.
- Заменить можно **не любое** выражение: макросы всегда выглядят, как переменные или как функции:

```
#define PI 3.1415926535
#define SUM(a, b) a + b
```

```
// До препроцессора:

double x = sin(PI);
int s = SUM(a, b);
```

```
// После препроцессора:

double x = sin(3.1415926535);
int s = a + b;
```

Макросы

- Если результат раскрытия макроса содержит макросы, они тоже раскрываются (но рекурсия запрещена):

```
#define PI 3.1415926535
#define SIN_DEG(x) sin(x / 180.0 * PI)
#define MY_ANGLE 30.0

double x = SIN_DEG(MY_ANGLE);

// Превратится в
double x = 30.0 / 180.0 * 3.1415926535;
```

Макросы: if

- Макросы можно использовать для условного включения кода:

```
#define USE_SIMD

// ...

#ifdef USE_SIND
  #include <immintrin.h>
#endif

vec4 add(vec4 a, vec4 b) {
  #ifdef USE_SIMD
    __m128 va = _mm_loadu_ps(&a.x);
    __m128 vb = _mm_loadu_ps(&b.x);
    __m128 vr = _mm_add_ps(va, vb);
    vec4 result;
    _mm_storeu_ps(&result.x, vr);
    return result;
  #else
    return (vec4){a.x + b.x, a.y + b.y, a.z + b.z, a.w + b.w};
  #endif
}
```

Макросы: if

- `#if` может содержать более сложные арифметические и логические выражения:

```
#if defined(__APPLE__) && LIB_VERSION >= 3
  ...
#endif
```

Макросы: if

- `#if` может содержать более сложные арифметические и логические выражения:

```
#if defined(__APPLE__) && LIB_VERSION >= 3
  ...
#endif
```

- Полезно для версионирования кода, определения окружения компилятора (ОС, битность системы), и т.п.

Макросы: проблемы

- Так как это просто текстовая подстановка (игнорирующая синтаксис C), в их использовании легко сделать ошибку

Макросы: проблемы

- Так как это просто текстовая подстановка (игнорирующая синтаксис C), в их использовании легко сделать ошибку

```c
#define MUL(a, b) a * b

int x = 5;
int y = MUL(x + 2, 7);

printf("%i", y);
```

Макросы: проблемы

- Так как это просто текстовая подстановка (игнорирующая синтаксис C), в их использовании легко сделать ошибку

```c
#define MUL(a, b) a * b

int x = 5;
int y = MUL(x + 2, 7);

printf("%i", y);
```

- Что выведется на экран?

Макросы: проблемы

- Так как это просто текстовая подстановка (игнорирующая синтаксис C), в их использовании легко сделать ошибку

```c
#define MUL(a, b) a * b

int x = 5;
int y = MUL(x + 2, 7);

printf("%i", y);
```

- Что выведется на экран? 19, так как код превратится в

  `x + 2 * 7`

Макросы: проблемы

- Как это исправить?

Макросы: проблемы

- Как это исправить?

```
#define MUL(a, b) (a) * (b)

int x = 5;
int y = MUL(x + 2, 7);

printf("%i", y);
```

Макросы: проблемы

- Как это исправить?

```c
#define MUL(a, b) (a) * (b)

int x = 5;
int y = MUL(x + 2, 7);

printf("%i", y);
```

- Теперь выведется 49, так как код превратится в `(x + 2) * (7)`

Макросы: проблемы

```c
#define ABS(x) ((x) >= 0 ? (x) : -(x))

int x = ABS(very_slow_function());

printf("%i", x);
```

Макросы: проблемы

```
#define ABS(x) ((x) >= 0 ? (x) : -(x))

int x = ABS(very_slow_function());

printf("%i", x);
```

- В чём проблема?

Макросы: проблемы

```
#define ABS(x) ((x) >= 0 ? (x) : -(x))

int x = ABS(very_slow_function());

printf("%i", x);
```

- В чём проблема? `very_slow_function()` вызовется два раза

Макросы: проблемы

```c
#define ABS(x) ((x) >= 0 ? (x) : -(x))

int x = 10;
int y = ABS(x++);

printf("%i %i", x, y);
```

Макросы: проблемы

```c
#define ABS(x) ((x) >= 0 ? (x) : -(x))

int x = 10;
int y = ABS(x++);

printf("%i %i", x, y);
```

- Что выведется на экран?

Макросы: проблемы

```c
#define ABS(x) ((x) >= 0 ? (x) : -(x))

int x = 10;
int y = ABS(x++);

printf("%i %i", x, y);
```

- Что выведется на экран? `12  11` (инкремент применится дважды)

Макросы: проблемы

```c
#define ABS(x) ((x) >= 0 ? (x) : -(x))

int x = 10;
int y = ABS(x++);

printf("%i %i", x, y);
```

- Что выведется на экран? `12  11` (инкремент применится дважды)
- **NB:** До C11 это UB, так как выражение содержит несколько побочных эффектов для одной и той же переменной

Макросы: проблемы

- Как исправить `ABS(x)`?

Макросы: проблемы

- Как исправить `ABS(x)`?
- Никак: лучше написать функцию

```
inline int abs(int x) {
  return x >= 0 ? x : -x;
}
```

Макросы: проблемы

```
#define SWAP(a, b) int temp = a; a = b; b = temp;
```

Макросы: проблемы

```
#define SWAP(a, b) int temp = a; a = b; b = temp;
```

- В чём проблема?

Макросы: проблемы

```
#define SWAP(a, b) int temp = a; a = b; b = temp;
```

- В чём проблема? Нельзя использовать в функции, где уже объявлена переменная `temp` (будет multiple definition):

```
int foo(int x) {
  int temp = x * x;
  SWAP(x, temp);
  return x;
}
```

Макросы: проблемы

- Как это исправить?

Макросы: проблемы

- Как это исправить? Явно создать вложенный скоуп (но лучше всё-таки функция):

```
#define SWAP(a, b) { int temp = a; a = b; b = temp; }
```

Макросы: проблемы

- Как это исправить? Явно создать вложенный скоуп (но лучше всё-таки функция):

```
#define SWAP(a, b) { int temp = a; a = b; b = temp; }
```

- Начиная с C23 можно не указывать тип:

```
#define SWAP(a, b) { typeof(a) temp = a; a = b; b = temp; }
```

Многострочные макросы

- Можно писать макросы в несколько строк, для этого в конце каждой строки кроме последней нужно поставить обратный слеш:

```
#define SWAP(a, b) { \
  typeof(a) temp = a; \
  a = b; \
  b = temp; }
```

Макросы: stringize

- Макросы позволяют превратить любое выражение в строку с помощью оператора препроцессора #:

```
#define STR(x) #x

printf(STR(x + 10));

// Превратится в
printf("x + 10");
```

Макросы: stringize

- Макросы позволяют превратить любое выражение в строку с помощью оператора препроцессора `#`:

```
#define STR(x) #x

printf(STR(x + 10));

// Превратится в
printf("x + 10");
```

- Раскрытия макросов внутри `x` при этом **не происходит**!

Макросы: stringize

- Полезно для логирования, assert'ов, unit-тестов, и т.п.:

```
#define ASSERT(x) if(!(x)) { \
    printf("Assertion failed: " #x "\n"); \
    exit(1); \
  }

int x = 10;
ASSERT(x < 0);
```

- Выведет:

```
Assertion failed: x < 0
```

Макросы: конкатенация

- Можно склеивать несколько выражений в одно с помощью оператора препроцессора `##`:

```
#define CONCAT(x, y) x##y

CONCAT(pri, ntf)("Hello, world!");

// Превратится в
printf("Hello, world!");
```

Макросы: конкатенация

- Можно склеивать несколько выражений в одно с помощью оператора препроцессора `##`:

```
#define CONCAT(x, y) x##y

CONCAT(pri, ntf)("Hello, world!");

// Превратится в
printf("Hello, world!");
```

- Раскрытия макросов внутри `x` и `y` при этом тоже **не происходит**!

Макросы: кодогенерация

- Конкатенацию можно использовать для генерации *generic* кода (для произвольных типов данных):

```
#define MAKE_MAX(type) type max_##type(type a, type b) { \
  return a > b ? a : b; }

MAKE_MAX(int)    // Сгенерирует max_int
MAKE_MAX(float)  // Сгенерирует max_float
MAKE_MAX(double) // Сгенерирует max_double

int x = max_int(10, 11);
double y = max_double(3.14, 2.71);
```

Макросы: кодогенерация

```
#define MAKE_ARRAY(type) typedef struct { \
  type * data; \
  size_t size; \
  } array_##type;

MAKE_ARRAY(int)

array_int a;
a.size = 10;
a.data = malloc(a.size * sizeof(*a.data));
```

Макросы: кодогенерация

```
#define MAKE_LIST(type) typedef struct list_##type##_t { \
  type value; \
  struct list_##type##_t * next; \
  } list_##type;

MAKE_LIST(int)

list_int l;
l.value = 10;
l.next = nullptr;
```

Макросы: упрощение кода

```
#define FOREACH(size, var) \
  for (int var = 0; var < size; ++var)

FOREACH(10, i) {
  printf("%d\n", i * i);
}
```

Макросы: упрощение кода

```
#define FOREACH(size, var) \
  for (int var = 0; var < size; ++var)

FOREACH(10, i) {
  printf("%d\n", i * i);
}
```

```
#define LIST_FOREACH(list, var) \
  for (node * var = list; var != NULL; var = var->next)

LIST_FOREACH(mylist, n)
  printf("%d\n", n->value);
```

Макросы: логирование

- Есть особые нестандартные, но широко распространённые макросы для получения информации о текущем месте в коде

Макросы: логирование

- Есть особые нестандартные, но широко распространённые макросы для получения информации о текущем месте в коде
- `__FILE__` раскрывается в имя файла, содержащего этот макрос

Макросы: логирование

- Есть особые нестандартные, но широко распространённые макросы для получения информации о текущем месте в коде
- `__FILE__` раскрывается в имя файла, содержащего этот макрос
- `__LINE__` раскрывается в номер строки в файле

Макросы: логирование

- Есть особые нестандартные, но широко распространённые макросы для получения информации о текущем месте в коде
- `__FILE__` раскрывается в имя файла, содержащего этот макрос
- `__LINE__` раскрывается в номер строки в файле

```
#define LOG(msg) printf("(%s:%d) " msg, __FILE__, __LINE__)

// В main.cpp, на 25-ой строке:
LOG("test");
```

- Выведет

```
(main.cpp:25) test
```

Макросы: variadic

- Макросы тоже могут быть variadic, т.е. иметь произвольное количество аргументов (обозначается тоже как многоточие …)

Макросы: variadic

- Макросы тоже могут быть variadic, т.е. иметь произвольное количество аргументов (обозначается тоже как многоточие `...`)
- `__VA_ARGS__` – макрос, раскрывающийся в список всех variadic аргументов (вместе с запятыми между ними)

Макросы: variadic

- Макросы тоже могут быть variadic, т.е. иметь произвольное количество аргументов (обозначается тоже как многоточие `...`)
- `__VA_ARGS__` – макрос, раскрывающийся в список всех variadic аргументов (вместе с запятыми между ними)

```
#define LOG(format, ...) printf(format, __VA_ARGS__)

LOG("%s %d", "Hello ", 10);
```

- Выведет

```
Hello 10
```

Макросы: variadic

- Если в variadic макрос не передать ни одного variadic аргумента, `__VA_ARGS__` раскрывается в пустоту

Макросы: variadic

- Если в variadic макрос не передать ни одного variadic аргумента, `__VA_ARGS__` раскрывается в пустоту
- Часто это становится проблемой, если передавать `__VA_ARGS__` аргументами в какую-то функцию – в конце остаётся лишняя запятая:

Макросы: variadic

- Если в variadic макрос не передать ни одного variadic аргумента, `__VA_ARGS__` раскрывается в пустоту
- Часто это становится проблемой, если передавать `__VA_ARGS__` аргументами в какую-то функцию – в конце остаётся лишняя запятая:

```
#define LOG(format, ...) printf(format, __VA_ARGS__)

LOG("Hello");

// Превратится в
printf("Hello", );
```

Макросы: variadic

- Решение: `__VA_OPT__` (C23, C++20)

Макросы: variadic

- Решение: `__VA_OPT__` (C23, C++20)
- `__VA_OPT__(X)` раскрывается в `X`, если список variadic аргументов непустой, иначе раскрывается в пустоту

Макросы: variadic

- Решение: `__VA_OPT__` (C23, C++20)
- `__VA_OPT__(X)` раскрывается в `X`, если список variadic аргументов непустой, иначе раскрывается в пустоту

```
#define LOG(format, ...) \
  printf(format __VA_OPT_(,) __VA_ARGS__)
```

Макросы: логирование с форматированием

```
#define LOG(format, ...) \
  printf("(%s:%d) " format, __FILE__, __LINE__ \
    __VA_OPT__(,) __VA_ARGS__)

int a = 10, b = 20;
// В main.cpp, в 25-ой строке
LOG("%d + %d = %d", a, b);
```

- Выведет

```
(main.cpp:25) 10 + 20 = 30
```